20%_PROCESS
Big Data Processing

Ton Duc Thang University

Nguyen Xuan Trieu[1], Pham Van Tien Dat[2], Nguyen Phuc Dat[3], Nguyen Van Truong[4], Nguyen Thanh An[5]
[1]521H0174, [2]521H0030, [3]521H0441, [4]521H0324
Ton Duc Thang University, Vietnam
[5]Lecturer, Ton Duc Thang University, Vietnam

***Abstract*—This project explores data mining techniques through two main tasks: Frequent Itemset Mining using the FPGrowth algorithm and Recommender Systems with the Alternating Least Squares (ALS) algorithm. Task 1 applies FPGrowth to discover frequent itemsets and generate association rules from the "baskets.csv" dataset. Task 2 focuses on using ALS to develop a recommendation system based on the "ratings2k.csv" dataset. Experimental results highlight the efficiency of FPGrowth in identifying patterns in transactional data and ALS in delivering personalized recommendations. The report also includes project progress, individual contributions, and a group performance review, providing a comprehensive analysis of both algorithms in real-world applications.**

***Index Terms*—Frequent Itemset Mining with FPGrowth, Recommender Systems with ALS**

I. Task 1: Frequent Item Set Mining – FPGrowth

A. *Task 1.1: Efficient Frequent Itemset Mining Using FP-Tree:*

Study and present the FPGrowth algorithm, in the PySpark library, for frequent item set mining. Present the algorithm in detail and given a simple example for explantion.

1) **Step 1: Count Item Frequencies**
   a) Count the frequency of each item.
   b) Remove items with frequency less than *min_support*.
   c) Sort the remaining items in descending order of frequency.
2) **Step 2: Build the FP-Tree**
   a) Create a root node (null).
   b) For each transaction:
      - Filter and rearrange the items based on the frequency order identified earlier.
      - Insert the items into the tree, sharing branches where possible.
3) **Step 3: Extract Frequent Patterns from the FP-Tree**
   a) Build a header table linking nodes of the same item.
   b) For each item (processed bottom-up):
      - Create the conditional pattern base.
      - Construct the conditional FP-Tree.
      - Identify frequent patterns.

Example: Data with minSupport = 2: Consider the following dataset and a minimum support of 2. The steps are illustrated below (results: Figure 1).

ID	Items
1	{phone, charger, case}
2	{phone, charger, cable}
3	{phone, case, headphone}
4	{charger, case}
5	{phone, headphone, charger}

Fig. 1: Input data with *minSupport = 2*

1) ***Step 1: Count Item Frequencies:***
   - Frequencies: Phone: 4; Charger: 4; Case: 3; Headphone: 2; Cable: 1.
   - Filtered items with *minSupport = 2*: L = {Phone, Charger, Case, Headphone}.
   - Items sorted by frequency (descending order).

ID	Items	Ordered-Itemset
1	{phone, charger, case}	{phone, charger, case}
2	{phone, charger, cable}	{phone, charger}
3	{phone, case, headphone}	{phone, case, headphone}
4	{charger, case}	{charger, case}
5	{phone, headphone, charger}	{phone, charger, headphone}

Fig. 2: Step 1: Frequency Count

2) ***Step 2: Build the FP-Tree: (Fig. 3):***
   - Create a root node (null).
   - Insert transactions into the tree:
   - Insert {*phone, charger, case*}.
   - Insert {*phone, case, headphone*}.
   - Insert {*charger, case*}.
   - Insert {*phone, charger, headphone*}.
3) ***Step 3: Extract Frequent Patterns:***
   - Create the Conditional Pattern Base. (Fig. 4)
   - Construct the Conditional FP-Tree. (Fig. 5)
   - Identify the Frequent Patterns. (Fig. 6)

B. *Task 1.2: Advantages, Disadvantages, and Case Studies*

a. Advantages:

- Faster and More Memory-Efficient:

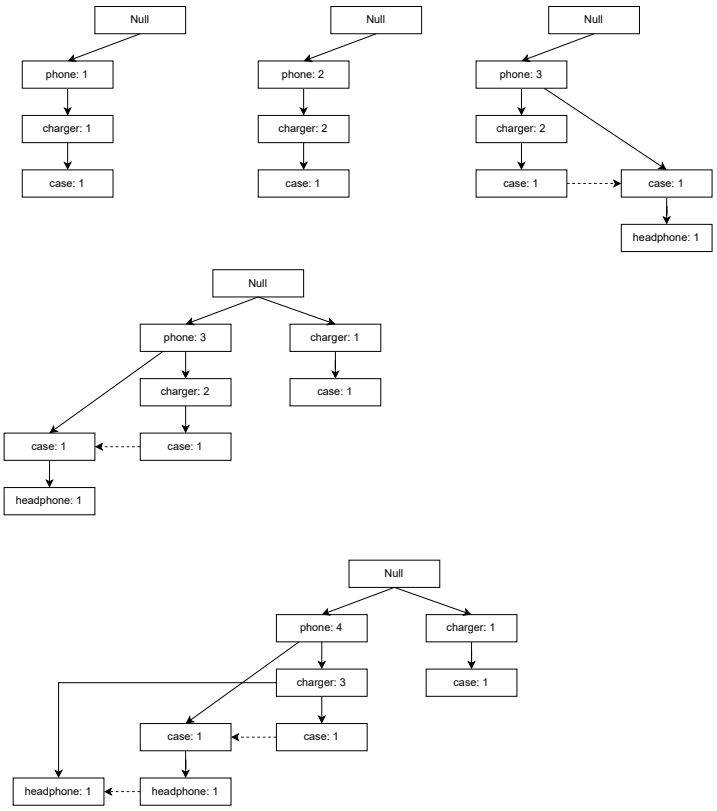


Fig. 3: Step 2 / Diagram of task 1

Items	Conditional Pattern Base
headphone	{{phone, charger: 1}, {phone, case: 1}}
case	{{phone: 1}, {phone, charger: 1}, {charger: 1}}
charger	{{phone: 1}}
phone	

Fig. 4: Step 3.1: Conditional Pattern Base

Items	Conditional Pattern Base	Conditional FP-Tree
headphone	{{phone, charger: 1}, {phone, case: 1}}	{phone: 2}
case	{{phone: 1}, {phone, charger: 1}, {charger: 1}}	{phone: 2, charger: 2}
charger	{{phone: 3}}	{phone: 3}
phone		

Fig. 5: Step 3.2: Conditional FP-Tree

Items	FP Generated
headphone	{<phone, headphone: 2>}
case	{<phone, case: 2>, <charger, case: 2>}
charger	{<phone, charger: 3>}
phone	

Fig. 6: Step 3.3: Frequent Patterns

  - FP-Growth outperforms algorithms like Apriori in terms of speed and memory usage, making it effective for processing datasets with a large number of items.
  - Because FP-Growth skips the costly process of generating and testing all possible item combinations and scans the dataset only twice, whereas Apriori requires multiple passes.
- Handles Large Datasets Efficiently:
  - The algorithm scales well with increasing data size and complexity, maintaining strong performance even with massive datasets.
  - Because this algorithm condenses large datasets into a compact FP-Tree, reducing memory usage and redundancy, and requires only two scans, it minimizes the time spent on large data processing.
- Focus on Relevant Patterns:
  - FP-Growth is resilient to noise in data because it filters out items that do not meet the minimum support threshold, focusing only on frequent and significant patterns.
  - By ignoring less frequent or irrelevant items, the algorithm ensures that it prioritizes meaningful associations, making it more efficient and reliable in handling noisy data. This approach allows FP-Growth to detect important patterns without being affected by outliers or rare occurrences.
- Easily Distributed:
  - FP-Growth is ideal for parallel and distributed computing because it divides the dataset into smaller partitions that can be processed independently across multiple machines or processors.
  - This parallel processing reduces computation time and allows the algorithm to scale efficiently with large datasets, making it well-suited for systems like Hadoop or Spark.

b. Disadvantages:

- Increased Memory Requirements for Large Datasets:
  - While FP-Growth is efficient with memory, it can consume significant resources as the dataset size grows because a higher number of unique items leads to a larger FP-tree, which requires more memory to store and process the data.
- Complex to Understand and Implement:
  - FP-Growth can be harder to grasp and apply compared to simpler methods for discovering frequent itemsets because it involves constructing and traversing an FP-tree, which may require a deeper understanding of tree structures and recursive algorithms.
- Slow with High Minimum Support:
  - When the minimum support threshold is set too high, the algorithm may slow down because it needs to process lengthy branches in the FP-tree, which in-

creases computational time as fewer patterns remain after the filtering.

- Not Ideal for Rare Pattern Detection:
  - FP-Growth is optimized for finding frequent patterns but is less suitable for identifying rare or infrequent patterns because it focuses on patterns that occur frequently, meaning it may overlook those that appear infrequently but still have significance.

c. Case Studies:

- Market Basket Analysis in Retail:
  - Identify product pairs frequently purchased together (e.g., bread and milk) to optimize product placement and promotions.
- Product Recommendation in E-Commerce:
  - Analyze shopping carts to suggest potential products customers may like, such as "People who buy phones often buy cases."
- Symptom Analysis in Healthcare:
  - Detect symptoms that commonly occur together to assist diagnosis (e.g., fever and muscle pain related to dengue fever).
- Movie Recommendation on Streaming Platforms:
  - Find movies often watched together to build a recommendation system, like Netflix or YouTube.
- Supply Chain Optimization:
  - Analyze groups of products frequently shipped together to optimize packaging and delivery routes.
- User Behavior Analysis:
  - Discover online shopping habits to display relevant ads, such as "Customers who buy cameras often purchase memory cards."

II. Task 2: Recommender System – ALS

Collaborative Filtering with Alternating Least Squares (ALS) is a popular method in recommendation systems, used to personalize user experiences on platforms like e-commerce sites and streaming services.
ALS iteratively optimizes to create a factorized representation of the data. The matrices are of $m \times n$, U $(m \times k)$, V $(n \times k)$.

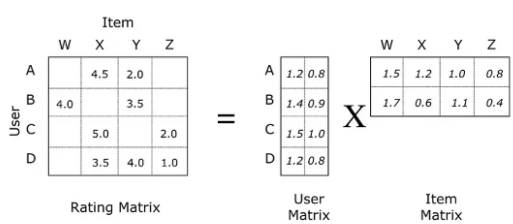


Fig. 7: Matrix Factorization

Equation:

$$R \approx X \times Y^T \tag{1}$$

Notation:

- R: Rating matrix $(m \times n)$
- X: User factors matrix $(m \times k)$
- Y: Item factors matrix $(n \times k)$
- m: Number of users
- n: Number of items
- k: Number of latent factors ($k < m$ and $k < n$)

By randomly initializing U and V, and applying iterative least squares, we find the optimal weights to approximate R. The least squares method minimizes the sum of squared differences between the data and the model's approximation.

Application: Used in collaborative filtering to generate personalized recommendations from user-item interactions.

Example: In e-commerce, ALS analyzes purchase history to recommend products based on patterns from similar users.

Using ALS with Spark

- **userCol:** Specifies the name of the column that contains the user indexes.
- **itemCol:** Specifies the name of the column that contains the item indexes (e.g., games).
- **ratingCol:** Specifies the name of the column that contains user ratings for the items.
- **coldStartStrategy:** Specifies the strategy for handling new users or items during forecasting. In this case, "drop" indicates that new users or items will be dropped.
- **nonnegative:** Indicates whether predictions should be restricted to non-negative values.

Hyperparameters

The following hyperparameters are specified for ALS:

- **rank:** Specifies the number of latent factors (also called dimensions) of the ALS.
- **maxIter:** Specifies the maximum number of iterations that ALS can run during training.
- **regParam:** Specifies the regularization term that controls the strength of the penalty to avoid overfitting.

Performance Metric

The Mean Squared Error (MSE) measures the average squared difference between predicted and actual values.

a. Advantages of ALS

- **Sparsity Handling:**
  The user-item matrix R in the recommendation system is often very sparse (most items have no reviews from users). ALS is designed to optimize based only on observations (non-zero entries as ratings), which saves memory and improves performance using sparse matrix representation.
- **No Product Information Required:**
  ALS is based solely on the interaction matrix between users and products, without the need for additional metadata (such as product descriptions). This makes it easy to deploy for datasets containing only ratings.

- **Supports Setting Explicit Hyperparameters:**
  The number of latent factors, regularization coefficients, and number of iterations can be easily adjusted to suit the problem.
- **Scalability:**
  ALS is designed to run well on distributed systems like Apache Spark. It divides work into smaller tasks that can be performed simultaneously, making it suitable for big data applications.

b. Disadvantages of ALS

- **Requires Low-Quality Data:**
  If the data is sparse but contains too few interactions (low-density matrix), ALS will have difficulty learning patterns from the data, leading to less accurate predictions.
- **Cold Start Problem:**
  ALS struggles to recommend for new users or items with no interaction data.
- **Hyperparameter Sensitivity:**
  The performance of ALS depends heavily on hyperparameters such as rank, regularization, and iterations, requiring careful tuning.
- **Incremental Updates Are Not Supported:**
  When new data is added (e.g., new users or new products), ALS cannot be updated incrementally. The entire matrix must be retrained, increasing the computational cost.

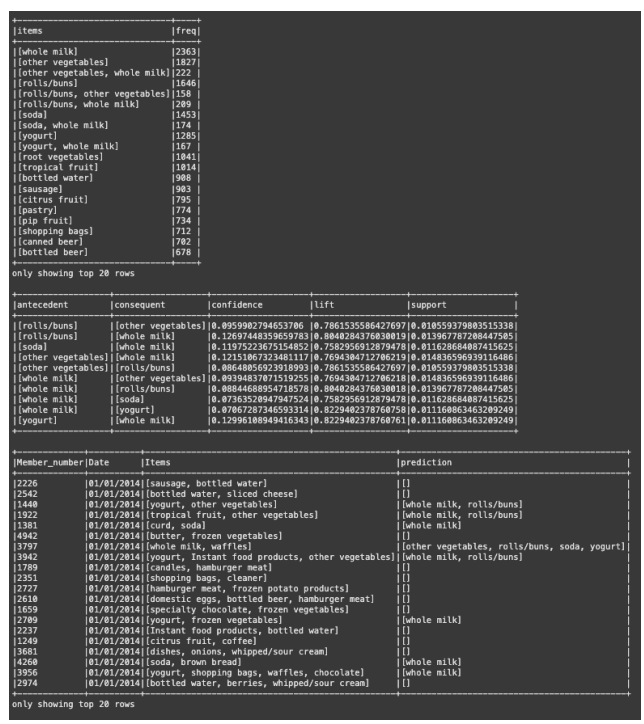
c. Case Studies

- **E-commerce (Amazon):**
  - **Use Case:** Recommending products based on past purchases and reviews.
  - **Outcome:** Improved customer engagement and higher sales through personalized suggestions.
- **Streaming Services (Netflix, Spotify):**
  - **Use Case:** ALS is employed to recommend movies, shows, or songs by analyzing viewing or listening history.
  - **Outcome:** Enhanced user satisfaction and retention by providing relevant content recommendations.
- **Online Learning Platforms (Coursera, Khan Academy):**
  - **Use Case:** Recommending courses based on learners' past activities and ratings.
  - **Outcome:** Increased course completion rates and user satisfaction.
- **Retail Analytics:**
  - **Use Case:** Understanding customer behavior by analyzing purchase patterns and recommending frequently bought items.
  - **Outcome:** Optimized inventory management and increased sales.
- **Travel and Hospitality (Booking.com, Airbnb):**
  - **Use Case:** Suggesting hotels or experiences based on user preferences and past bookings.
  - **Outcome:** Higher booking rates and customer loyalty.

III. Use FPGrowth and ALS classes in PySpark

Frequent itemsets from `baskets.csv` using FPGrowth

1) **Initialization**: Define FPGrowthModel with the following parameters:
   - **baskets_path**: Path to the baskets data file.
   - **min_support**: Minimum support threshold.
   - **min_confidence**: Minimum confidence threshold.
2) **Main Methods**:
   - **discover_baskets**: Prepares transaction data for analysis.
   - **fit_model**: Trains the FP-Growth algorithm on the prepared data.
   - **display_frequent_itemsets**: Displays the frequent itemsets discovered by the model.
   - **display_association_rules**: Shows the generated association rules.
   - **apply_model**: Applies the model to make predictions.
3) **Execution**: The process involves creating an object, preparing data, training the model, displaying results, and applying the model.
4) **Result: (see more in output folder)**



```
+------------------------------+----+
|items                         |freq|
+------------------------------+----+
|[whole milk]                  |2363|
|[other vegetables]            |1827|
|[other vegetables, whole milk]|222 |
|[rolls/buns]                  |1646|
|[rolls/buns, other vegetables]|158 |
|[rolls/buns, whole milk]      |209 |
|[soda]                        |1453|
|[soda, whole milk]            |174 |
|[yogurt]                      |1285|
|[yogurt, whole milk]          |167 |
|[root vegetables]             |1041|
|[tropical fruit]              |1014|
|[bottled water]               |908 |
|[sausage]                     |903 |
|[citrus fruit]                |795 |
|[pastry]                      |774 |
|[pip fruit]                   |734 |
|[shopping bags]               |712 |
|[canned beer]                 |702 |
|[bottled beer]                |678 |
+------------------------------+----+
only showing top 20 rows

+------------------+------------------+--------------------+--------------------+--------------------+
|antecedent        |consequent        |confidence          |lift                |support             |
+------------------+------------------+--------------------+--------------------+--------------------+
|[rolls/buns]      |[other vegetables]|0.095990279465370 6 |0.7861535586427697|0.010559379803515338|
|[rolls/buns]      |[whole milk]      |0.1269744835965978 3|0.8040284376030019|0.013967787208447505|
|[soda]            |[whole milk]      |0.1197522367515485 2|0.7582956912879478|0.011628684087415625|
|[other vegetables]|[whole milk]      |0.1215106732348117 |0.7694304712706219|0.014836596939116486|
|[other vegetables]|[rolls/buns]      |0.0864005692391893 |0.7861535586427697|0.010559379803515338|
|[whole milk]      |[other vegetables]|0.0939483707151925 5|0.7694304712706218|0.014836596939116486|
|[whole milk]      |[rolls/buns]      |0.0884468805471857 8|0.8040284376030018|0.013967787208447505|
|[whole milk]      |[soda]            |0.0736352094794752 4|0.7582956912879478|0.011628684087415625|
|[whole milk]      |[yogurt]          |0.0706728734659331 4|0.8229402378760758|0.011160863463209249|
|[yogurt]          |[whole milk]      |0.1299610894941634 3|0.8229402378760761|0.011160863463209249|
+------------------+------------------+--------------------+--------------------+--------------------+

+-------------+----------+-----------------------------------------------------+-----------------------------------------------------+
|Member_number|Date      |Items                                                |prediction                                           |
+-------------+----------+-----------------------------------------------------+-----------------------------------------------------+
|2226         |01/01/2014|[sausage, bottled water]                             |[]                                                   |
|2542         |01/01/2014|[bottled water, sliced cheese]                       |[]                                                   |
|1440         |01/01/2014|[yogurt, other vegetables]                           |[whole milk, rolls/buns]                             |
|1922         |01/01/2014|[tropical fruit, other vegetables]                   |[whole milk, rolls/buns]                             |
|1381         |01/01/2014|[curd, soda]                                         |[whole milk]                                         |
|4942         |01/01/2014|[butter, frozen vegetables]                          |[]                                                   |
|3797         |01/01/2014|[whole milk, waffles]                                |[other vegetables, rolls/buns, soda, yogurt]         |
|3942         |01/01/2014|[yogurt, Instant food products, other vegetables]    |[whole milk, rolls/buns]                             |
|1789         |01/01/2014|[candles, hamburger meat]                            |[]                                                   |
|2351         |01/01/2014|[shopping bags, cleaner]                             |[]                                                   |
|2727         |01/01/2014|[hamburger meat, frozen potato products]             |[]                                                   |
|2610         |01/01/2014|[domestic eggs, bottled beer, hamburger meat]        |[]                                                   |
|1659         |01/01/2014|[specialty chocolate, frozen vegetables]             |[]                                                   |
|2709         |01/01/2014|[yogurt, frozen vegetables]                          |[whole milk]                                         |
|2237         |01/01/2014|[Instant food products, bottled water]               |[]                                                   |
|1249         |01/01/2014|[citrus fruit, coffee]                               |[]                                                   |
|3681         |01/01/2014|[dishes, onions, whipped/sour cream]                 |[]                                                   |
|4260         |01/01/2014|[soda, brown bread]                                  |[whole milk]                                         |
|3956         |01/01/2014|[yogurt, shopping bags, waffles, chocolate]          |[whole milk]                                         |
|2974         |01/01/2014|[bottled water, berries, whipped/sour cream]         |[]                                                   |
+-------------+----------+-----------------------------------------------------+-----------------------------------------------------+
only showing top 20 rows
```

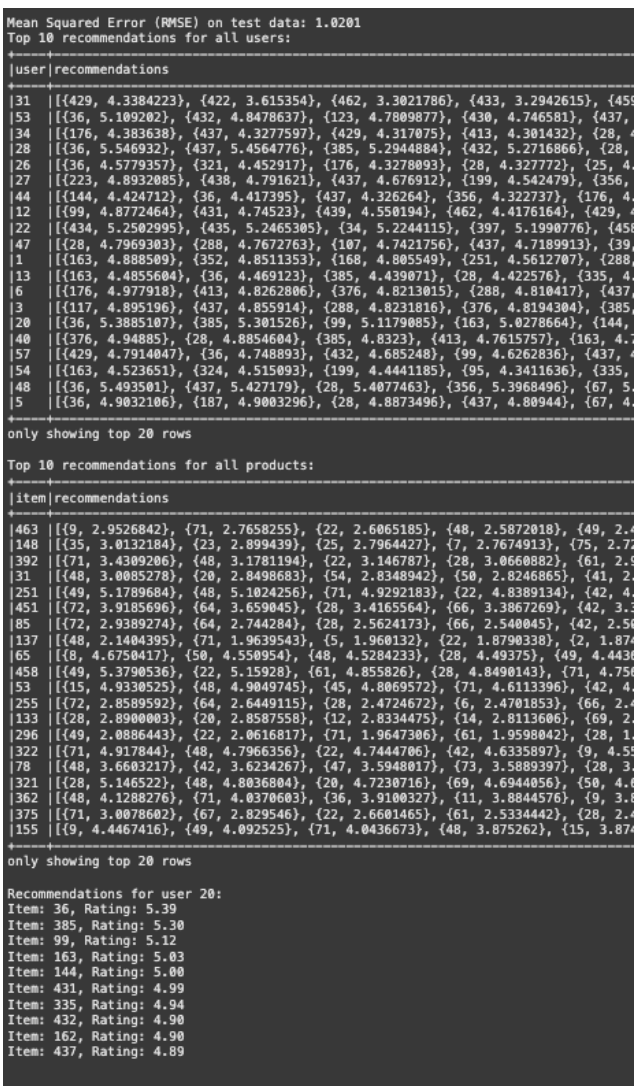
Using the ALS class with `ratings2k.csv`

1) **Initialization**: Define `ALSModel`, which handles data management, ALS model training, and recommendation generation.

2) **Main Methods**:

- **load_data** and **split_data**: Load the dataset and split it into training and test sets.
- **train_model**: Trains the ALS (Alternating Least Squares) model.
- **evaluate_model**: Evaluates the model's performance using RMSE.
- **product_recommendation** and **user_recommendation**: Generate product and user-based recommendations.
- **print_recommendations**: Displays the generated recommendations.

3) **Execution**: The `run` method automates the entire process, from data preparation to displaying results.
4) **Result: (see more in output folder)**



```
Mean Squared Error (RMSE) on test data: 1.0201
Top 10 recommendations for all users:
+----+--------------------------------------------------------------------------------
|user|recommendations
+----+--------------------------------------------------------------------------------
|31  |[{429, 4.3384223}, {422, 3.615354}, {462, 3.3021786}, {433, 3.2942615}, {459
|53  |[{36, 5.109202}, {432, 4.8478637}, {123, 4.7809877}, {430, 4.746581}, {437,
|34  |[{176, 4.383638}, {437, 4.3277597}, {429, 4.317075}, {413, 4.301432}, {28, 4
|28  |[{36, 5.546932}, {437, 5.4564776}, {385, 5.2944884}, {432, 5.2716866}, {28,
|26  |[{36, 4.5779357}, {321, 4.452917}, {176, 4.3278093}, {28, 4.327772}, {25, 4.
|27  |[{223, 4.8932085}, {438, 4.791621}, {437, 4.676912}, {199, 4.542479}, {356,
|44  |[{144, 4.424712}, {36, 4.417395}, {437, 4.326264}, {356, 4.322737}, {176, 4.
|12  |[{99, 4.8772464}, {431, 4.74523}, {439, 4.550194}, {462, 4.4176164}, {429, 4
|22  |[{434, 5.2502995}, {435, 5.2465305}, {34, 5.2244115}, {397, 5.1990776}, {458
|47  |[{28, 4.7969303}, {288, 4.7672763}, {107, 4.7421756}, {437, 4.7189913}, {39,
|1   |[{163, 4.888509}, {352, 4.8511353}, {168, 4.805549}, {251, 4.5612707}, {288,
|13  |[{163, 4.4855604}, {36, 4.469123}, {385, 4.439071}, {28, 4.422576}, {335, 4.
|6   |[{176, 4.977918}, {413, 4.8262806}, {376, 4.8213015}, {288, 4.810417}, {437,
|3   |[{117, 4.895196}, {437, 4.855914}, {288, 4.8231816}, {376, 4.8194304}, {385,
|20  |[{36, 5.3885107}, {385, 5.301526}, {99, 5.1179085}, {163, 5.0278664}, {144,
|40  |[{376, 4.94885}, {28, 4.8854604}, {385, 4.8323}, {413, 4.7615757}, {163, 4.7
|57  |[{429, 4.7914047}, {36, 4.748893}, {432, 4.685248}, {99, 4.6262836}, {437, 4
|54  |[{163, 4.523651}, {324, 4.515093}, {199, 4.4441185}, {95, 4.3411636}, {335,
|48  |[{36, 5.493501}, {437, 5.427179}, {28, 5.4077463}, {356, 5.3968496}, {67, 5.
|5   |[{36, 4.9032106}, {187, 4.9003296}, {28, 4.8873496}, {437, 4.80944}, {67, 4.
+----+--------------------------------------------------------------------------------
only showing top 20 rows

Top 10 recommendations for all products:
+----+--------------------------------------------------------------------------------
|item|recommendations
+----+--------------------------------------------------------------------------------
|463 |[{9, 2.9526842}, {71, 2.7658255}, {22, 2.6065185}, {48, 2.5872018}, {49, 2.4
|148 |[{35, 3.0132184}, {23, 2.899439}, {25, 2.7964427}, {7, 2.7674913}, {75, 2.72
|392 |[{71, 3.4309206}, {48, 3.1781194}, {22, 3.146787}, {28, 3.0660882}, {61, 2.9
|31  |[{48, 3.0085278}, {20, 2.8498683}, {54, 2.8348942}, {50, 2.8246865}, {41, 2.
|251 |[{49, 5.1789684}, {48, 5.1024256}, {71, 4.9292183}, {22, 4.8389134}, {42, 4.
|451 |[{72, 3.9185696}, {64, 3.659045}, {28, 3.4165564}, {66, 3.3867269}, {42, 3.3
|85  |[{72, 2.9389274}, {64, 2.744284}, {28, 2.5624173}, {66, 2.540045}, {42, 2.56
|137 |[{48, 2.1404395}, {71, 1.9639543}, {5, 1.960132}, {22, 1.8790338}, {2, 1.874
|65  |[{8, 4.6750417}, {50, 4.550954}, {48, 4.5284233}, {28, 4.49375}, {49, 4.4436
|458 |[{49, 5.3790536}, {22, 5.15928}, {61, 4.855826}, {28, 4.8490143}, {71, 4.756
|53  |[{15, 4.9330525}, {48, 4.9049745}, {45, 4.8069572}, {71, 4.6113396}, {42, 4.
|255 |[{72, 2.8589592}, {64, 2.6449115}, {28, 2.4724672}, {6, 2.4701853}, {66, 2.4
|133 |[{28, 2.8900003}, {20, 2.8587558}, {12, 2.8334475}, {14, 2.8113606}, {69, 2.
|296 |[{49, 2.0886443}, {22, 2.0616817}, {71, 1.9647306}, {61, 1.9598042}, {28, 1.
|322 |[{71, 4.917844}, {48, 4.7966356}, {22, 4.7444706}, {42, 4.6335897}, {9, 4.55
|78  |[{48, 3.6603217}, {42, 3.6234267}, {47, 3.5948017}, {73, 3.5889397}, {28, 3.
|321 |[{28, 5.146522}, {48, 4.8036804}, {20, 4.7230716}, {69, 4.6944056}, {50, 4.6
|362 |[{48, 4.1288276}, {71, 4.0370603}, {36, 3.9100327}, {11, 3.8844576}, {9, 3.8
|375 |[{71, 3.0078602}, {67, 2.829546}, {22, 2.6601465}, {61, 2.5334442}, {28, 2.4
|155 |[{9, 4.4467416}, {49, 4.092525}, {71, 4.0436673}, {48, 3.875262}, {15, 3.874
+----+--------------------------------------------------------------------------------
only showing top 20 rows

Recommendations for user 20:
Item: 36, Rating: 5.39
Item: 385, Rating: 5.30
Item: 99, Rating: 5.12
Item: 163, Rating: 5.03
Item: 144, Rating: 5.00
Item: 431, Rating: 4.99
Item: 335, Rating: 4.94
Item: 432, Rating: 4.90
Item: 162, Rating: 4.90
Item: 437, Rating: 4.89
```

IV. Conclusion

This project successfully applied two data mining techniques: FP-Growth and ALS. The FP-Growth algorithm efficiently identified frequent itemsets and generated association rules from the shopping cart data. Meanwhile, the ALS-based recommender system provided accurate personalized suggestions for users. Both methods have wide practical applications, from market basket analysis to online services like e-commerce and entertainment.

TABLE I: **Table of Tasks Complete Percentages**

Task ID	Full Name	Complete Percentage
1	Task 1 (4.0 point(s)): Frequent Item Set Mining – FPGrowth	100%
2	Task 2 (4.0 point(2)): Recommender Systems – ALS	100%
3	Task 3 (2.0 point(s)): Report	100%

TABLE II: **Table of Task Assignment Summary**

Student ID	Full Name	Email	Assigned Tasks	Complete Percentage
521H0174	Nguyen Xuan Trieu	521H0174@student.tdtu.edu.vn	1, 2, 3	100%
521H0030	Pham Van Tien Dat	521H0030@student.tdtu.edu.vn	1, 2, 3	100%
521H0441	Nguyen Phuc Dat	521H0441@student.tdtu.edu.vn	1, 2, 3	100%
521H0324	Nguyen Van Truong	521H0324@student.tdtu.edu.vn	1, 2, 3	100%

Acknowledgment

We would like to express our gratitude to Mr. Nguyen Thanh An for his dedicated teaching and for imparting valuable knowledge about Big Data, which enabled us to complete this "20% PROCESS report".

References